

The 15-Day Agenda

PHASE 1 — Python & Engineering Foundation (Days 1–3)

Day 1 — Python Foundations for AI Engineering

Objective: Get everyone to a shared, working Python baseline.

- Environment: Python, virtual envs (venv/uv), pip, VS Code, notebooks
- Core syntax: variables, types, control flow, comprehensions
- Data structures: lists, dicts, sets, tuples; iteration patterns
- Functions: args/kwargs, lambdas, intro to decorators
- Files & JSON I/O; modules, packages and imports
- **Lab:** Build a small CLI utility that reads, transforms and writes JSON.

Day 2 — Applied Python for AI

Objective: The specific Python that agentic engineering depends on.

- OOP and dataclasses
- **Type hints & Pydantic** — models and validation (the backbone of structured outputs and tool contracts later)
- Robust error handling and retry logic
- HTTP & REST: requests/httpx, auth headers, pagination
- async/await, asyncio and concurrency basics
- Environment variables and secrets (.env)
- **Lab:** Build an async API client that calls a public API and validates responses with Pydantic.

Day 3 — Engineering Practices + AI-Assisted Coding + First LLM App

Objective: Professional workflow, then the first LLM call — the bridge into the agentic phase.

- Git & GitHub: branching, commits, pull requests
- Project structure, dependency management, logging
- Testing with pytest: unit tests, fixtures, parametrization
- **AI-assisted coding foundations:** GitHub Copilot / Claude Code — prompting the assistant, reviewing and securing generated code

- First LLM call: OpenAI/Azure OpenAI SDK, prompts, JSON/structured output, function/tool calling
 - **Lab:** A small, tested Python project plus a tool-calling LLM script, generated with AI assistance and human-reviewed.
-

PHASE 2 — Core Agentic Building Blocks (Days 4–8)

Day 4 — Generative AI & Agentic AI Foundations + ADLC

Objective: The conceptual and engineering foundation for enterprise agents.

- Foundation models/LLMs: capabilities, limitations, context windows, tokens, inference, model selection
- Enterprise GenAI architecture: models, tools, knowledge, orchestration, memory, guardrails
- Structured outputs, function/tool calling, deterministic patterns
- Agents vs chatbots vs workflows vs autonomous systems; agent components (reasoning, planning, tools, memory, state, policies, feedback)
- Architectures: single-agent, supervisor, planner-executor, multi-agent; lifecycle: perceive → plan → act → observe → evaluate → recover
- **ADLC introduction:** business framing → design → build → validate → deploy → monitor → improve; Definition of Done and acceptance criteria
- **Lab:** Compare a conventional LLM workflow vs an agentic workflow for a service-desk case; create a business problem canvas (objective, users, constraints, KPIs, AI/non-AI boundaries). **Select the evolving enterprise use case used for the rest of the program.**

Day 5 — Prompt Engineering & Context Engineering

Objective: Systematic prompt/context design, taught once from basics to advanced.

- Prompt anatomy: role, objective, context, constraints, examples, output contract
- Core techniques: zero/few-shot, role prompting, decomposition, reasoning-oriented, critique/refinement, structured output
- Advanced architectures: system-prompt layering & policy hierarchy; task/routing/critic/evaluator prompts; modular reusable components; prompt contracts

- Context engineering: selection/prioritization/relevance scoring; compression & progressive disclosure; working vs long-term memory; managing large context windows without dilution; context boundaries across handoffs
- Tool-use & reasoning control: tool selection instructions and preconditions; tool-result validation; confidence-aware routing/escalation; controlled self-correction
- Prompt security: direct & indirect injection; instruction-hierarchy conflicts; data-exfiltration and tool-abuse; hardening and adversarial prompt test design
- **Lab:** Build prompt patterns (classification, extraction, reasoning, JSON); refactor a complex agent prompt stack for reliability/security/token efficiency; build an adversarial prompt suite and compare baseline vs hardened behavior.

Day 6 — Enterprise RAG

Objective: Grounded, secure retrieval over enterprise knowledge.

- RAG vs fine-tuning; when each is preferable
- Ingestion, chunking, metadata, embeddings, vector search
- Hybrid retrieval, reranking, context assembly
- Grounded generation, citations, retrieval quality, hallucination reduction
- RAG security: access control, data leakage, source trust; retrieval-failure handling
- **Lab:** Build an enterprise RAG assistant with ingestion, retrieval and grounded responses over the use-case corpus.

Day 7 — MCP, Tools & Agent Building Blocks

Objective: Connect agents to enterprise systems safely.

- MCP concepts: hosts, clients, servers, tools, resources, prompts
- MCP vs direct API integration; tool abstraction
- Connecting agents to enterprise systems; least-privilege tool access, permissions and security
- Designing reusable MCP-enabled capabilities
- Single-agent tool-calling patterns; agent roles (planner, researcher, retriever, executor, reviewer, supervisor)
- **Lab:** Build an MCP server exposing a controlled enterprise-style tool, connect it to an agent, and build a single tool-calling agent for the use case.

Day 8 — LangGraph & Advanced Multi-Agent Orchestration

Objective: From basic graphs to production-grade multi-agent workflows.

- LangGraph: graphs, nodes, edges, state, conditional routing; checkpointing & resumable workflows; HITL interrupts; controlled termination; observability hooks
 - Topologies: sequential, parallel, conditional, hierarchical
 - Advanced patterns: supervisor hierarchies & dynamic delegation; planner-executor-critic loops; fan-out/fan-in, map-reduce, quorum; nested workflows & reusable subgraphs; dynamic routing by confidence/cost/risk
 - Handoff protocol engineering: typed handoff contracts, structured inter-agent messages, context ownership & minimization, handoff validation/rejection/re-routing; avoiding loops, deadlocks and uncontrolled delegation
 - HITL at scale: risk-based approval policies; approval queues, escalation, timeouts; override/resume/audit
 - Workflow decomposition: deterministic vs agentic vs human steps; autonomy boundaries; invariants, pre/postconditions, termination
 - **Lab:** Build a LangGraph multi-agent workflow (planner → researcher → reviewer → response); add conditional routing, typed handoffs, parallel execution with aggregation, and a human approval checkpoint.
-

PHASE 3 — Hardening, Scale & Quality (Days 9–13)

Day 9 — Fault Tolerance, Resilience & Recovery Engineering

Objective: Engineer for failure rather than treating failures as exceptions.

- Failure taxonomy: model/tool/retrieval/state/orchestration failures; transient vs permanent; partial success & inconsistent downstream state; failure propagation
- Recovery: retry budgets, exponential backoff, retry classification; compensation & semantic rollback; checkpoint recovery & resumability; dead-letter & failed-task replay; fallback across models/tools/agents
- Reliability patterns: circuit breakers, bulkheads, concurrency controls; timeout hierarchies & cancellation propagation; idempotency keys & duplicate-execution prevention; exactly-once vs at-least-once; graceful degradation & safe-stop

- **Failure-injection lab:** inject model/tool/retrieval/downstream failures; measure recovery; build a resilience scorecard and remediation backlog.

Day 10 — Cloud-Native Architecture & Scaling on Azure AI Foundry

Objective: Move from prototype to enterprise-scale cloud architecture.

- Azure AI Foundry: concepts, model catalog, deployments, endpoints; project/environment architecture; model routing & endpoint abstraction; agent/tool integration & enterprise identity; dev/test/prod separation
- Production architecture: API/orchestration/model/knowledge layers + enterprise systems; stateless vs stateful services and persistent state stores
- Scalability: horizontal scaling of stateless services; async workloads, queues, event-driven execution; long-running/durable execution; concurrency, rate limits, back-pressure
- Enterprise integration: API management & service boundaries; data/system integration; managed identities, secrets, access policies; network isolation & private connectivity
- Cost & operational architecture: token economics & workload cost modeling; model selection by task/latency/cost; caching & context reuse; availability, disaster recovery, capacity planning
- **Lab:** Design a production reference architecture (topology, service boundaries, security controls, scaling strategy) and deploy the containerized multi-agent app to an Azure AI Foundry-style environment.

Day 11 — Specs-Driven Development & AI-Assisted Engineering at Scale

Objective: Specifications as the bridge between business intent, agent behaviour, code, tests and evidence.

- Specs hierarchy: business requirement → functional spec → agent behaviour spec
- Agent contracts, tool contracts, state contracts
- Behavioural acceptance criteria and executable specifications
- Traceability across requirements, code, tests and evaluations
- AI-assisted engineering at scale: repository-aware development; architecture-to-code & spec-to-code; AI-assisted refactoring/modernization; generating tests/docs/artifacts; human review & secure coding controls

- Workflow patterns: plan → specify → implement → test → evaluate → review → release; PR quality gates & review checklists; managing AI-generated technical debt
- **Lab:** Convert a business requirement into a spec; derive agent/tool contracts and acceptance tests; generate an agent implementation from the spec with an AI assistant; establish requirement → code → test → evaluation traceability.

Day 12 — Agent Evaluation & Testing Engineering

Objective: A rigorous, automated quality discipline (evaluation + testing taught together, since they share tooling).

- **Evaluation architecture:** objectives mapped to business outcomes; component vs workflow level; offline/online/pre-release; golden, edge-case and adversarial datasets
- **Metrics:** task success/workflow completion; groundedness/relevance/completeness/consistency; tool-call precision, argument correctness, tool-selection accuracy; latency/cost/reliability; safety/policy adherence
- **Methodology:** LLM-as-judge design, calibration and limitations; human evaluation & inter-rater considerations; thresholds & release criteria; baseline management & regression detection; evidence & traceability
- **Testing strategy:** test pyramid adapted for LLM/agents; unit/component/workflow/integration/system/acceptance; deterministic vs probabilistic design; test data, fixtures, synthetic generation
- **Agent-specific tests:** prompt regression; tool selection/argument; handoff/routing validation; memory/state persistence; RAG grounding & retrieval-failure; human-approval/escalation paths
- **Adversarial & safety testing:** injection/jailbreak; unsafe tool use & excessive agency; data leakage & sensitive context; boundary and malformed input
- **Test automation:** CI execution; parameterized suites; regression baselines & failure triage; evidence capture for release
- **Lab:** Build a production-style evaluation dataset and automated eval runs comparing model/prompt/workflow versions; build an end-to-end agent testing harness (positive, negative, edge, adversarial, failure-recovery); integrate into CI.

Day 13 — Observability, Debugging & Production Operations (Langfuse)

Objective: Observability as an engineering feedback loop.

- Deep observability with Langfuse: trace hierarchy across agents/generations/tools/steps; token/latency/cost observability; prompt/version tracking & experiment comparison; metadata, tags, correlation IDs; feedback & production outcomes
- Debugging methodology: trace-first workflow; root-cause across prompt/model/retrieval/tool/orchestration; detecting context pollution and bad handoffs; latency/cost hotspots; failure clustering & recurring defects
- Production monitoring: operational/quality/safety/business signals; alert thresholds & anomaly detection; drift indicators & regression monitoring; incident investigation & post-incident learning
- **Lab:** Instrument the multi-agent workflow with Langfuse; analyze traces to find quality/latency/cost bottlenecks; implement improvements and compare before/after evidence.

PHASE 4 — Governance & Delivery (Days 14–15)

Day 14 — AI Risk, Threat Modeling, Governance & CI/CD Delivery Gates

Objective: Operationalize risk management as an engineering discipline and delivery gate.

- Advanced risk analysis: risk taxonomy across models/agents/tools/data/users; hallucination & unsupported-action; model drift & behavior change; bias/fairness & unintended outcomes; data integrity, stale knowledge, provenance
- Agent threat modeling: for autonomous/tool-using agents; trust boundaries & attack surfaces; injection, tool misuse, privilege escalation; data exfiltration & indirect attacks; third-party model/service dependency
- Risk controls: guardrails & policy enforcement; tool authorization & least privilege; input/output validation; human approval for high-impact actions; safe-stop, escalation, audit
- CI/CD validation gates: automated prompt/model/workflow regression tests in CI; safety/security checks before release; evaluation thresholds & release-blocking criteria; canary/rollback/post-release monitoring; traceability from specs to evaluation evidence
- **Lab:** Build an AI risk register and threat model for the use case; map each risk to preventive/detective/corrective controls; implement a CI/CD validation gate that blocks deployment when thresholds fail.

Day 15 — Delivery Metrics, Production Readiness & Enterprise Capstone

Objective: Bring engineering, quality, governance and operations together into a production-delivery framework.

- Advanced ADLC operating model: embedding validation/risk/testing into every stage; Definition of Ready / Definition of Done for agentic apps; architecture/quality/safety review checkpoints; production acceptance & operational handover
- Lead-level metrics: **MTTV** (time from validated problem to business value); **AI CFR** (change/release failure rate); leading vs lagging indicators; connecting engineering metrics to business outcomes; using metrics to prioritize
- Production readiness: quality/safety/reliability/scalability/observability/cost assessment; operational runbooks & incident response; rollback & model/version management; monitoring baselines & improvement backlog
- **Enterprise simulation & capstone review:** review the end-to-end solution against architecture/testing/evaluation/security/risk/operational criteria; execute release-gate decisions using evidence; run failure/incident scenarios; present business impact, risks, controls, metrics and a production recommendation.